
KV CACHE OPTIMIZATION STRATEGIES FOR SCALABLE AND EFFICIENT LLM INFERENCE

Yichun Xu¹ , Navjot K. Khaira² , and Tejinder Singh² 

¹Dell Technologies, Hopkinton, MA 01748, USA

²Dell Technologies, Santa Clara, CA 95054, USA

{Yichun.Xu, Navjot.Khaira, Singh.Tejinder}@Dell.com

March 24, 2026

ABSTRACT

The key-value (KV) cache is a foundational optimization in Transformer-based large language models (LLMs), eliminating redundant recomputation of past token representations during autoregressive generation. However, its memory footprint scales linearly with context length, imposing critical bottlenecks on GPU memory capacity, memory bandwidth, and inference throughput as production LLMs push context windows from thousands to millions of tokens. Efficient KV cache management has thus become a first-order challenge for scalable LLM deployment. This paper provides a systematic review of recent KV cache optimization techniques, organizing them into five principal directions: cache eviction, cache compression, hybrid memory solutions, novel attention mechanisms, and combination strategies. For each category we analyze the underlying mechanisms, deployment trade-offs, and empirical performance across memory reduction, throughput, and model accuracy metrics. We further map techniques to seven practical deployment scenarios, including long-context single requests, high-throughput datacenter serving, edge devices, multi-turn conversations, and accuracy-critical reasoning, providing actionable guidance for practitioners selecting among competing approaches. Our analysis reveals that no single technique dominates across all settings; instead, the optimal strategy depends on context length, hardware constraints, and workload characteristics, pointing toward adaptive, multi-stage optimization pipelines as a promising direction for future research.

Keywords Key-Value Cache Management · LLMs Efficiency · Transformer Memory Optimization · Attention Mechanism Compression · Memory Bandwidth Optimization

1 Introduction

KV cache optimization is crucial for LLM deployment. It reduces latency by avoiding recomputing past tokens as shown in Fig 1, lowers operational costs with less compute requirements, and enables deployment on resource-constrained devices by managing large memory footprints and optimizing data transfers. Recent advancements in LLMs illustrate a rapid and industry-wide increase in supported context window sizes. In recent years, modern models across different vendors have expanded from tens of thousands of tokens to hundreds of thousands, and in some cases millions. This reflects a broader trend rather than the evolution of any single product line, underscoring the growing importance of efficient KV-cache management in large context inference.

Unlike individual research papers that typically focus on one KV-cache optimization method in isolation, or prior surveys that provide broad but shallow coverage, this survey offers a middle-ground perspective. Our work systematically reviews and categorizes recent strategies for KV cache optimization into five major directions: cache eviction, compression and reconstruction, hybrid memory solutions, novel attention mechanisms, and combined approaches. Each category presents unique trade-offs between memory efficiency, computational cost, and model accuracy. Representative methods such as H₂O, SnapKV, and Ada-KV demonstrate the effectiveness of intelligent token selection [1, 2, 3, 4], while compression techniques like KIVI highlight the potential of quantization [5].

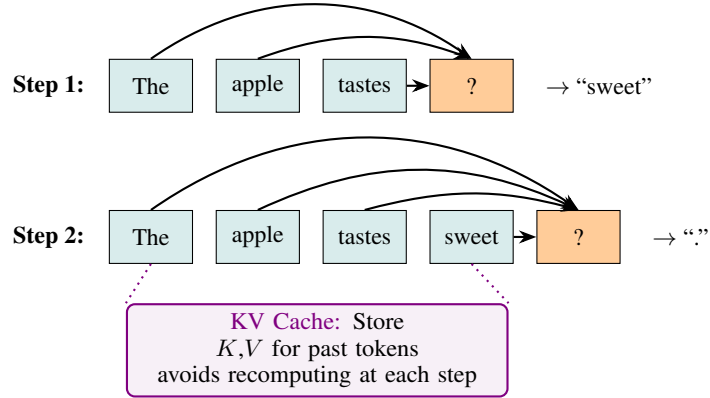


Figure 1: Autoregressive generation, at each step the new token (orange) attends to all prior tokens (cyan). Without caching, keys and values for every past token would be recomputed from scratch at each step. The KV cache avoids this by storing and reusing them.

This paper focuses on the most recent approaches in each category, providing detailed explanations of their mechanisms rather than simply brief mentions. This approach ensures clarity and depth, enabling readers to understand the principles behind each method and their implications for future research.

2 Background

2.1 Context Length

The concept of a fixed input sequence length (context length) was first introduced in Attention Is All You Need [6]. The authors proposed the Transformer architecture and defined the usage of a fixed-size input sequence length due to computational constraints of the self-attention mechanism. The term “context length” and “context window” subsequently became standard terminology in the machine learning to refer to this fixed limit on the number of tokens a model could process at one time. For instance, a model with a context length of 2,048 tokens can attend to up to 2,048 tokens at once. This parameter is typically documented in model cards and varies across architectures. Because attention requires storing keys and values for every past token, increasing context length directly inflates KV cache size and bandwidth demands. As a result, larger context lengths impose higher memory consumption and communication overhead, especially in distributed inference settings.

2.2 KV Cache

The key value (KV) cache is an inherent optimization of the Transformer architecture. It stores intermediate key and value vectors from previous tokens to avoid redundant computation during generation. The key vector represents the identity of a token, while the value vector represents the actual content of that token. As new tokens are generated, their corresponding keys and values are appended to the existing cache. In this way, KV cache can help reduce latency and computational overhead. Fig. 2 illustrates this data-flow within a single transformer layer.

While KV caching accelerates inference, its memory footprint grows linearly with context length, since each token in the context is represented by a key value pair. For a Transformer model, the KV cache size can be expressed as:

$$KV_{per\ token} = 2 \times H \times D \times B \times L \quad (1)$$

where H is the number of attention heads; D the dimension of each head; L the number of transformer layers; B the bytes per element (typically 4 for float32). Since these parameters are fixed for a given model, the KV per token is constant for a model.

$$KV_{cache\ size} = KV_{per\ token} \times ContextLength \quad (2)$$

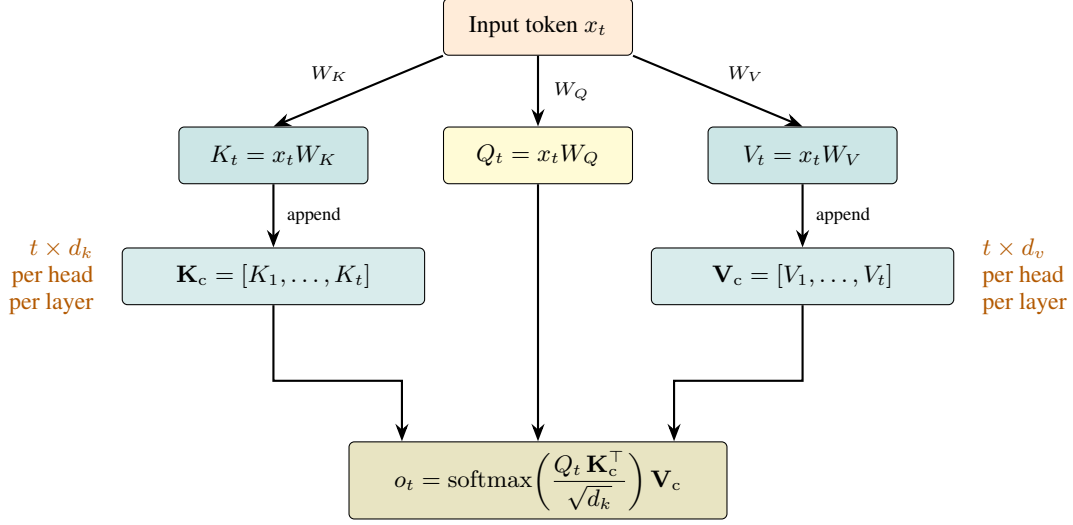


Figure 2: Data-flow of the KV cache within a single transformer layer. Input token x_t fans into three projections; K_t and V_t are appended to their respective caches (teal); Q_t attends over the full caches to produce output o_t . Cache size grows as $O(T)$ per head per layer.

With modern LLMs extending context windows from thousands to millions of tokens, this linear growth imposes significant memory and bandwidth demands, making KV cache optimization essential for efficient deployment, as quantified in Fig. 3 for three representative model sizes.

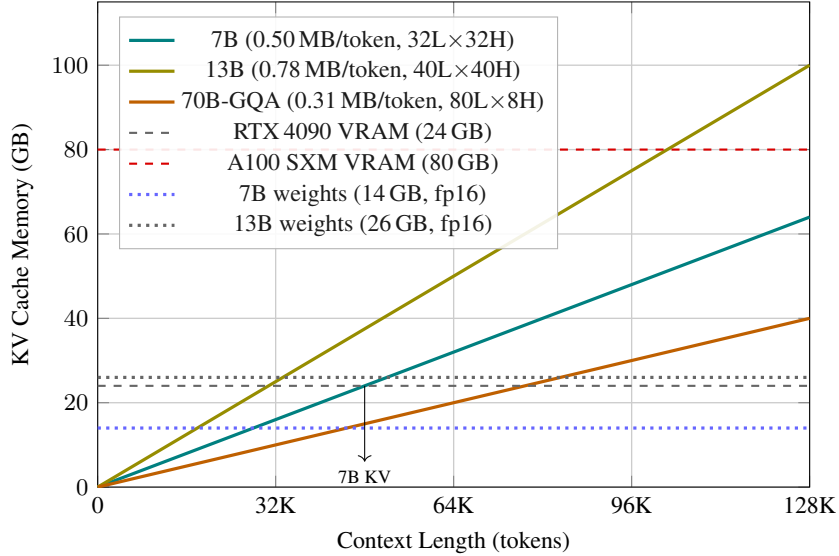


Figure 3: KV cache memory as a function of context length for three LLaMA-2 model variants under fp16 precision. Dashed lines mark GPU VRAM limits; dotted lines mark model parameter memory. At 128K tokens, a 7B model’s KV cache (≈ 64 GB) exceeds the capacity of an A100 GPU, illustrating the memory bottleneck that motivates KV cache optimization. Values computed as $2 \times L \times H_{kv} \times d_h \times 2$ bytes per token; 70B uses GQA with 8 KV heads.

2.3 Attention Score

The Transformer architecture relies on an attention mechanism to determine the relevance of each token in a sequence when generating the next token. Attention score is calculated to quantify how much a token should attend to other tokens in the input. For example, when generating the word “sweet” in the sentence “The apple tastes sweet”,

the model assigns higher attention to “apple” than to “The”, as it is more contextually relevant. The most widely used approach for computing attention scores is Scaled Dot-Product Attention.

For a query vector Q_i corresponding to token i and a key vector K_j for token j , the similarity is computed as:

$$\text{Score}(Q_i, K_j) = \frac{Q_i \cdot K_j^T}{\sqrt{d_k}} \quad (3)$$

where d_k is the dimension of the hidden layer, also known as the size of vectors used in the model. Dot product represents how much two vectors point in the same direction. In the above formula, dot product of Q_i and K_j^T shows the similarity of i ’s query is to j ’s key, reflecting how much token i attend to previous token j during generation.

These scores are normalized using the softmax function to produce attention weights that sum to 1:

$$\alpha_{ij} = \text{softmax}\left(\frac{Q_i \cdot K_j^T}{\sqrt{d_k}}\right) \quad (4)$$

Finally, the output for token i is computed as a weighted sum of the value vectors V_j :

$$\text{output}_i = \sum_j \alpha_{ij} V_j \quad (5)$$

This process repeats for each new token during generation, enabling the model to incorporate contextual information efficiently. Fig. 4 provides a concrete numerical example of Eqs. (3)–(5) for the sentence “The apple tastes sweet,” highlighting the non-uniform attention distribution that motivates selective KV cache eviction.

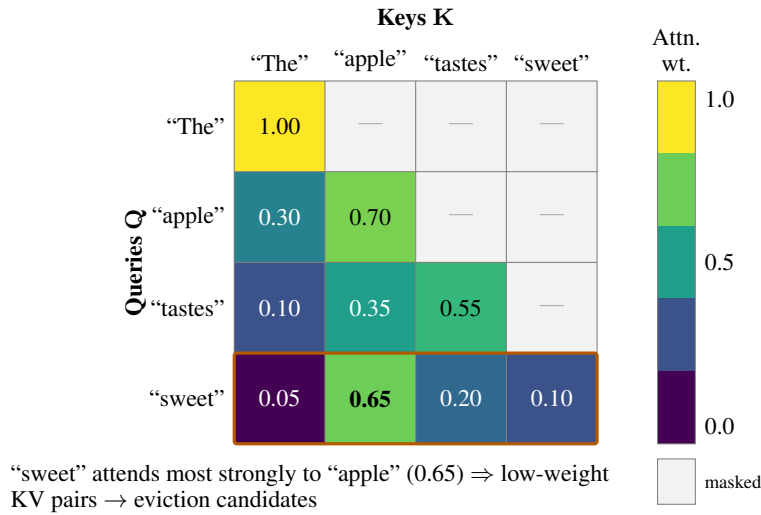


Figure 4: Causal self-attention weight matrix for “The apple tastes sweet.” visualised with the Viridis colormap (dark purple = low, yellow = high). Gray cells are causally masked future tokens. Each row sums to 1 (post-softmax). Query “sweet” concentrates 65% of its attention on “apple”, demonstrating that KV entries carry highly non-uniform importance, the core premise of attention-score-driven eviction methods such as H₂O and SnapKV.

2.4 Taxonomy

To provide a structured overview, we present a taxonomy that groups existing methods into five major categories shown in Fig. 5. Each category is characterized by the system-level objectives it aims to improve alongside the trade-offs it introduces Table 1. This taxonomy offers readers a high-level comparison of design motivations and limitations across approaches, serving as a reference point for understanding how different methods prioritize performance, efficiency, and model quality.

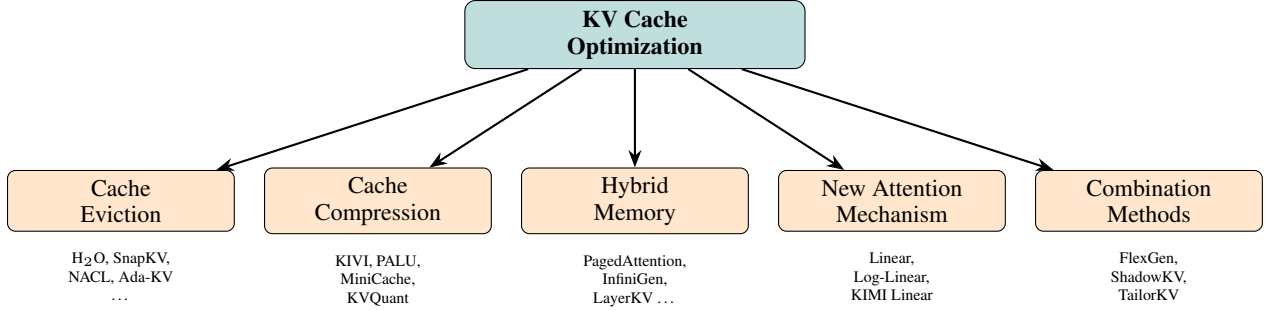


Figure 5: Taxonomy of KV cache optimization techniques surveyed in this paper, organized into five major categories.

Table 1: Comparison of KV Cache optimization techniques

Technique	Optimization Goal	Tradeoffs	Representative Methods	Good for
Cache Eviction	Memory Footprint, Decoding Latency	Accuracy and quality	H ₂ O, SnapKV, NACL, InfiniPot, Hasheviot, MorphKV, RocketKV, KVzip, Ada-KV	Long-context single requests, Edge devices, Minimal model modification
Cache Compression	KV cache size, Throughput	Dequantization and reconstruction overhead, Accuracy	KIVI, MiniCache, PALU, KVQuant	Bandwidth-bound workloads, Edge devices, Ultra-long contexts
Hybrid memory solution	TTFT in long-context, System efficiency	Hardware requirement, complexity in management	PagedAttention, InfiniGen, LayerKV, INF2, KVPR, Oneiros, CLO	High-throughput serving, Multi-tenant datacenter serving
New Attention Calculation	computational complexity, Inference Speed	Accuracy, model retraining	Linear, Log-Linear, Local Linear, KIMI Linear	Agentic/Long-horizon Tasks
Combination Methods	Throughput/Latency Balance	Complexity in design and management	FlexGen, Q-Hitter, ShadowKV, TailorKV	Consumer-Grade Hardware and Diverse Workloads, balanced performance

3 Content

As discussed earlier, increasing context length puts significant pressure on GPU memory and memory bandwidth, thus slowing down token throughput. While modern LLMs continue to expand their context window sizes to support more complex tasks, recent models have reached millions of tokens. [7, 8] This trend further amplifies the memory and computational challenges associated with KV cache management.

To address these challenges, both academia and industry have proposed a range of optimization strategies. This survey reviews and categorizes these approaches into five major directions: (1) cache eviction methods that selectively discard less critical tokens, (2) compression and reconstruction techniques that reduce memory footprint, (3) hybrid memory solutions leveraging multi-tier storage, (4) novel attention mechanisms that rethink context processing, and (5) combination strategies that integrate multiple optimizations.

3.1 Cache Eviction

Cache eviction is a strategy that selectively discards the less important cache while retaining the critical ones for accurate generation. This method usually relies on attention scores to estimate the token importance. Tokens with high scores are retained while those with low scores are discarded. This method maintains KV cache in a manageable size to reduce memory footprint. The summary of all the methods described in this section is given in Table 2.

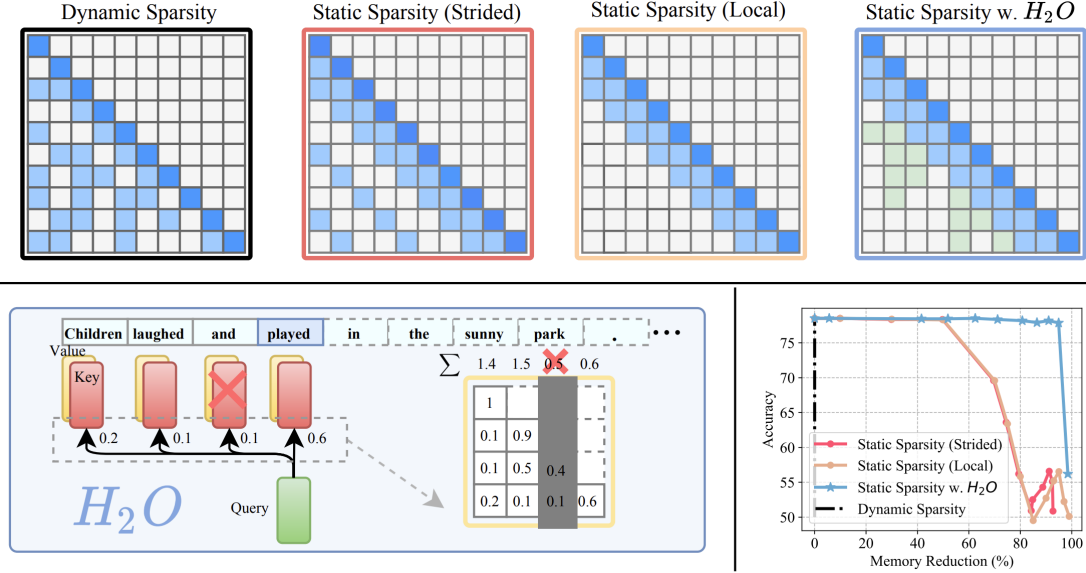


Figure 6: Upper plots illustrate symbolic plots of an attention map deploying different KV cache policies in LLM generation. Lower right: contrasts their accuracy-memory trade-off. Left: the overview of H_2O framework [1].

A key challenge in eviction-based methods is identifying which tokens carry long-range importance. One approach tracks accumulated attention scores and treats high-scoring tokens as essential for maintaining model accuracy. Work by Zhengyu et al. [1] shown in Fig. 6 denotes the tokens with high accumulated attention scores as heavy-hitter (H2) tokens. Losing the H2 tokens leads to significant performance degradation during generation. To address this, the authors propose H_2O (Heavy-Hitter Oracle), a dynamic eviction policy that balances retention of recent tokens with heavy-hitter tokens. The researchers used a greedy algorithm to manage a fixed cache size budget. The algorithm starts with an empty cache. For each new token, if the cache has space, the token will be added to the cache. Otherwise, the system computes the accumulated attention scores for each token in the cache, including the new token. The token with the lowest score will be discarded to maintain the cache size.

Another method addresses token selection during the encoding phase. SnapKV [2] introduced a token selection and compression strategy designed for long-context prompts. SnapKV applies a voting mechanism within the observation window to identify the most important previous tokens. An observation window is the most recent segment of tokens in the input. It is used to calculate attention scores for each previous token. The scores are subsequently aggregated to identify the most influential tokens across the entire sequence. To preserve contextual coherence, SnapKV employs 1D pooling to cluster nearby important tokens to “retain the features surrounding the selected attention features” [2]. In other words, SnapKV retains not only the top K important tokens, but also the context surrounding them. During the compression phase, SnapKV keeps both the selected important tokens from the prefix and all tokens in the observation window. This approach enables efficient KV cache compression for long prompts while maintaining high accuracy.

In contrast to step-wise eviction, the work in NACL [9] adopts a single-shot KV cache eviction strategy that combines Proxy-Tokens Eviction with Random Eviction. Proxy tokens are a small subset within the input, that “are responsible for yielding the most precise outcomes during the computation of the token score” [9]. Usually, NACL picks the few tokens at the end of the input, such as the user’s question, as the proxy tokens. A scoring function calculates the accumulated attention scores of all other tokens against only these selected proxy tokens, providing a precise measurement of tokens’ importance in the task-relevant context. The less important tokens will be discarded. To further optimize cache size, NACL incorporates Random Eviction. The token selection is based on a probability distribution, rather than pure randomness. The attention scores calculated between proxy tokens and all other tokens are turned into a probability distribution using SoftMax. Tokens are then sampled from this distribution to fill the remaining eviction budget. The selected tokens are discarded all-together during encoding, leading to one big eviction, rather than one by one. Compared to traditional methods (like H_2O introduced in previous section) that discard tokens one by one repeatedly, this approach reduces computation stress.

Some recent research explores continual distillation as a strategy for managing long contexts. The approach in [10] uses novelty measurements to retain essential information when memory limits are reached. The authors take an analogy of InfiniPot to be like a cooking pot: when a pot nears overflow, it distills unnecessary parts and retains only the essentials.

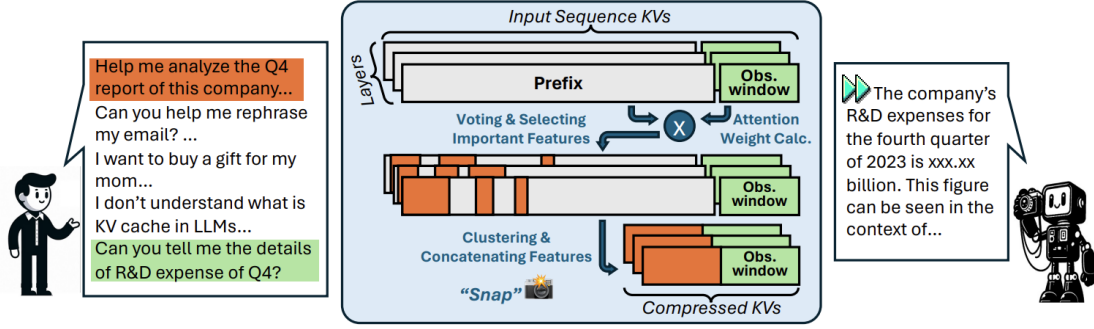


Figure 7: The graph shows the simplified workflow of SnapKV, where the orange area represents the cluster of features per head selected by SnapKV. These features are then used to form new Key-Value pairs concatenated with the features in the observation window. Together, the selected prefix and observation windows constitute the new KV cache utilized for the generation. [2].

InfiniPot enables pre-trained LLMs to handle “infinitely” long contexts within fixed memory constraints by introducing Continual Context Distillation (CCD). Token importance is determined with two metrics: CaP (Catalyst Prompt) and NuC (Novelty under Compression). CaP measures how representative a token is for future context. The system adds a short prompt like “Summarize the critical points” before the pot overflows and then computes attention scores to see which tokens matter most. NuC measures how new or unique a token is compared to existing ones. Tokens that are harder to predict will get higher NuC scores. Before memory saturation, the system triggers a distillation step. Based on the CaP and NuC scores, the system keeps the top K tokens and distills the rest. This dynamic distillation ensures that critical and novel information remains accessible while maintaining a constant memory footprint.

A lightweight alternative avoids full attention computation altogether. HASHEVICT [11] introduces a pre-attention KV cache eviction strategy that leverages Locality-Sensitive Hashing (LSH) to estimate token similarity before performing full attention computation, thereby reducing both memory and computational overhead. Specifically, it uses SimHash, an LSH algorithm where similar data points produce similar hash codes. In HASHEVICT, each token’s key embedding and the current token’s query embedding is converted into a binary hash code using random projections. Token similarity is then approximated by computing the Hamming distance between these hash codes. As shown in Section 2.9.2 of [12], hamming distance is the number of positions where two codewords of the same length differ. Here it refers to the number of differing bits between the two binary hash codes. Tokens with the largest Hamming distance (least similarity) are assumed to have low attention relevance and are evicted. This method is efficient and lightweight because the operations used for hashing and hamming distance calculation are simple and GPU friendly. Therefore, by enabling a single round of token eviction prior to the actual attention calculation, HASHEVICT helps to reduce both memory and computational cost.

Another technique focuses on leveraging recent attention patterns. The framework presented in MorphKV [13] dynamically selects relevant old tokens based on aggregated or peak attention signals. To be more specific, it uses attention patterns of recent tokens to decide which older tokens remain relevant. MorphKV only preserves the most relevant old tokens and a fixed number of recent tokens to limit KV cache size. The system selects the important old tokens by scoring for each old token with either Sum Fusion or Max Fusion function. Both of them are based on Softmax attention scores. Sum Fusion aggregates attention scores across recent tokens, favoring tokens consistently attended by multiple recent tokens. It is ideal for maintaining contextual consistency. In contrast, Max Fusion selects tokens based on the highest individual attention score, making it suitable for tasks requiring sharp, focused retrieval. If at least one recent token strongly cares about a particular old token, it implies that the old token is of high importance under Max fusion. Therefore, by dynamically selecting tokens based on these relevance measures, MorphKV is able to keep a constant KV cache size while preserving accuracy.

A two-stage design is explored in RocketKV [14], which combines coarse eviction with dynamic sparse selection to accelerate long-context inference without significant accuracy loss. RocketKV uses SnapKV [14, 2] to do coarse-grain KV cache eviction in the first stage and uses Hybrid Sparse Attention (HAS) to select relevant tokens dynamically in the second stage. HAS groups tokens into pages and stores the maximum and minimum key values for each page, enabling fast approximation of token importance page wise. Based on these approximations, the system retrieves the original KV pairs for the selected pages and uses them for attention computation. This hierarchical approach balances efficiency and accuracy, significantly improving throughput for large-context LLMs.

Table 2: Summary of KV Cache eviction techniques

Method	Mechanism	Phase	Overview
H ₂ O [1]	Evict top K noncritical tokens	Decoding	Retains a balance of Heavy-Hitter tokens (H2) and the most recent tokens
SnapKV [2]	Retain critical tokens and their context	After Prefill	Identifies critical attention features by voting from observation window and uses pooling for clustering
NACL [9]	Hybrid Proxy and Random Eviction	Prefill	Performs a single, global eviction during encoding
InfiniPot [10]	Continual Context Distillation	Prefill	Enables processing of infinite context within a fixed memory budget
HASHEVICT [11]	Attention-Free LSH Ranking	Decoding	Uses Locality-Sensitive Hashing (LSH) and Hamming Distance to estimate token importance
MorphKV [13]	Correlation-aware selection	Decoding	Maintains a constant-sized cache for long-response tasks while eliminating early-token bias
RocketKV [14]	Two-stage compression	Prefill and Decoding	Two-stage compression: permanent coarse eviction (as in SnapKV) followed by fine-grain dynamic selection (HSA)
KVzip [15]	Context reconstruction	Prefill	Importance scoring using maximum cross-attention
Ada-KV [3]	Adaptive Budget Allocation	After Prefill	Dynamically allocates cache budget across attention heads rather than a fixed budget per head

A reconstruction-driven and query-agnostic KV cache eviction approach is introduced in KVzip [15], where token importance is derived from a self-supervised context reconstruction process rather than from query-specific signals. During the initial context encoding, the model processes a full input context and generates a complete KV cache. To assess token significance, KVzip performs a self-supervised reconstruction task by prompting the model with instructions such as “Repeat the previous content” followed by the original context. This reconstruction simulates an autoencoder-like mechanism, where the model attempts to regenerate the input using only the cached KV pairs. During this process, the model’s attention mechanism naturally reveals the importance of KV pairs. Tokens that contribute most to accurate reconstruction are deemed critical, while those with lower impact are evicted. This approach effectively identifies essential KV pairs without relying on query-specific information, enabling efficient compression while preserving contextual integrity.

In traditional cache eviction methods, there exists a key limitation; that is the fixed cache budget is distributed uniformly across all attention heads while ignoring head-specific attention patterns. To address this issue, Yuan et al. proposed Ada-KV [3], a head-wise adaptive budget allocation strategy. The core idea of this Ada-KV approach is to minimize the eviction loss by reallocating budget from “attention-sparse heads” (where weights are concentrated on few elements) to “attention-dispersed heads” (with widespread concentration patterns). The authors establish a theoretical framework for cache eviction by defining eviction loss and deriving its upper bound [3]. Each attention head dynamically retains a variable number of critical tokens based on its attention distribution, rather than a fixed amount to every head like the traditional methods do. Furthermore, Ada-KV is designed as a plug-and-play solution that can seamlessly integrate with other methods, such as SnapKV, to enhance overall efficiency.

3.2 Cache Compression

Cache compression aims to reduce memory footprint. The most common approach is quantization, which represents numerical values with fewer bits. The common format for modern transformer models to store activations during inference is fp16, also known as 16-bit floating point [16]. Quantization can reduce the size to a 4-bit integer [17, 18]. Along with cache quantization, other compression strategies target structural redundancies, such as layer-wise or matrix-level compression, enabling further optimization of storage and computational efficiency as given in the summary form in Table 3.

One line of compression research focuses on asymmetric quantization. The method proposed in KIVI [5] applies per-channel and per-token quantization schemes to preserve precision while reducing memory. KIVI stands for Key-Value cache with Input-dependent quantization. The authors observe that for key cache, “there are a few fixed channels

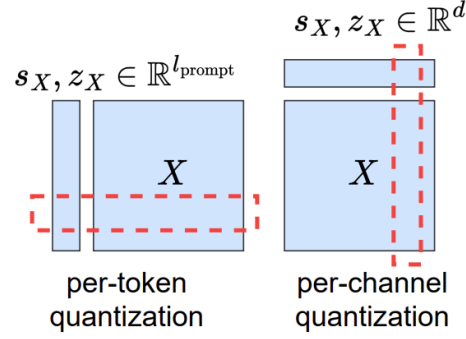


Figure 8: Definition of per-token and per-channel quantization. $X \in \mathbb{R}^{l_{\text{prompt}} \times d}$ is the key/value cache, where l_{prompt} is the number of tokens and d is the number of channels. z_X is the zero-point, and s_X is the scaling factor.. [5].

whose magnitudes are very large”; whereas for value cache, “there is no obvious outlier pattern”. Based on this insight, KIVI applies per-channel quantization for keys and per-token quantization for values, preserving precision where it matters most. To illustrate, if one channel contains large values and another contains small values, quantizing them separately maintains accuracy compared to uniform quantization. For example, if we have 3 tokens and 2 channels as:

Token1: [10, 0.5] Token2: [12, 0.6] Token3: [15, 0.5]

Since the first channel has large values and the second channel has small values, we can quantize the values based on different channels to keep each channel’s precision. That gives us the first channel (10, 12, 15) to be 0, 1, 3, and the second channel (0.5, 0.6, 0.5) to be 0, 3, 0.

Additionally, KIVI organizes the KV cache into groups (e.g., 32 tokens per group) and residuals (recent tokens kept in full precision). The group will be quantized to save memory, while the residuals will be kept in full precision for accuracy. As generation progresses, residuals are merged into groups after quantization, ensuring a balance between efficiency and performance. For key cache, this happens every time the number of residuals reaches the pre-defined threshold length. For value cache, this happens every time a new token is generated. This dynamic grouping mechanism allows KIVI to achieve significant memory savings without compromising inference quality.

While most of the conventional methods focus on redundancy within a single layer, MiniCache introduces a cross-layer KV cache compression technique that exploits redundancy between adjacent layers. The researchers observed that KV cache states show high similarity between adjacent layers in the middle to deep layers, as the representation start to stabilize in those layers. With this observation, this method begins by measuring similarity between KV states of neighboring layers using angular distance (or cosine similarity). If the similarity falls below a predefined threshold, the two layers’ KV states are considered mergeable. To preserve information during merging, MiniCache employs SLERP (Spherical Linear Interpolation) rather than direct averaging. After merging, it stores the interpolated direction, the

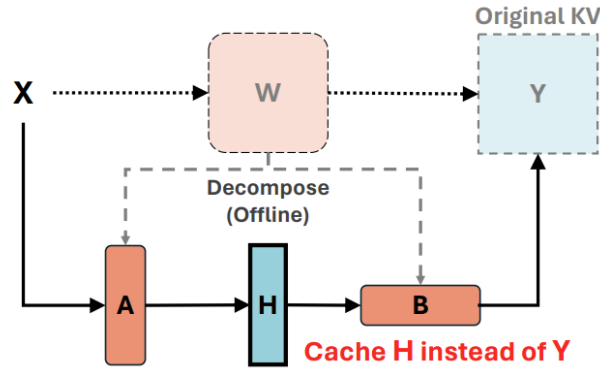


Figure 9: Palu’s low-rank projection method for KV-cache reduction. A weight matrix $\mathbf{W}$ of linear projection is decomposed into two low-rank matrices. Input $\mathbf{X}$ is down-projected to a latent representation $\mathbf{H}$, which is cached. $\mathbf{Y}$ can be reconstructed from $\mathbf{H}$ using the up-projection matrix $\mathbf{B}$. [19].

original magnitudes of both original vectors, and the angle between the vectors, enabling accurate reconstruction when needed by rescaling the stored direction. This approach significantly reduces memory usage while maintaining the ability to restore original KV states for attention computation.

Low-rank projection has also been explored to compress the KV cache. With the system described in PALU [19], each projection matrix W for both keys and values are decomposed into two smaller matrices A and B Via Singular Value Decomposition (SVD), such that $W \approx A \times B$. For example, if the input token vector X has a size of 4096 and we know each attention head uses 128 dimensions, the projection matrix W will have a size of 4096×128 . Matrix W can be factorized into matrix A ($4096 \times r$) and matrix B ($r \times 128$), where r is a smaller value than 128, for example, 32. SVD mathematically finds the best low-rank approximation of W , meaning that $A \times B$ is close to the original W .

During inference, the system multiplies input X (size = 4096) by A (size = $4096 \times r$) to obtain a latent representation H (size r). Instead of caching the original KV, only the small sized H is cached. When reconstruction is needed, multiplying H by B restores the original KV representation. To minimize runtime overhead, Palu uses Matrix Fusion to optimize computing costs during reconstruction. Palu fuses the reconstruction matrix (matrix B) into other existing weight matrices offline before inference, avoiding separate reconstruction steps. However, for Keys that use Rotational Positional Embedding (RoPE), matrix fusion is not possible, so PALU reconstructs the Key dynamically using a custom and highly efficient GPU kernel. Group-Head Low-Rank Decomposition (G-LRD) is another optimization method used. Joint decomposition across all heads provides high accuracy, yet per-head decomposition allows low reconstruction cost. To balance accuracy and cost, PALU groups several attention heads together and performs decomposition on that group.

Ultra-low-bit quantization is taken further in KVQuant [20], which integrates per-channel quantization, pre-RoPE processing, non-uniform quantization, and outlier preservation to enable LLM inference with context lengths up to 10 million tokens. The method combines the following key components:

Per-Channel Key Quantization: Similar to KIVI, keys are quantized per channel, instead of per token, to save memory and improve accuracy.

Pre-RoPE Quantization: RoPE is a rotation that LLMs apply to keys. This rotation makes quantization harder. Pre-RoPE Key Quantization applies quantization on keys before RoPE rotation is applied. This method simplifies computation and maintains structural integrity for higher accuracy.

Sensitivity-Weighted Non-Uniform Quantization: Compared to traditional uniform quantization that evenly spaces the quantization levels, KVQuant uses Sensitivity-Weighted Non-Uniform Quantization, where the quantization levels are optimized based on the data distribution to handle the outliers effectively. For example, for a key vector [0.02, 0.03, 0.05, 0.07, 10.0]. For uniform quantization into 3 bits ($2^3 \rightarrow 8$ levels), we evenly space the quantization levels to get levels: [-10, -7.5, -5, -2.5, 0, 2.5, 5, 7.5, 10]. In this case, all small numbers (0.02–0.07) map to 0, losing precision. In contrast, using non-uniform quantization gives us [0, 0.02, 0.04, 0.06, 0.08, 1, 5, 10]. Now small numbers keep details, and big outliers still fit.

Per-Vector Dense-and-Sparse Quantization : Extreme outliers (1% of elements) are stored separately in higher precision, while the majority of values are quantized to low bit, balancing memory savings and accuracy.

Table 3: Cache Compression Methods Comparison Table

Method	Mechanism	Granularity	Outlier handle
KIVI [5]	Asymmetric Quantization	Key: Per-channel; Value: Per-token	Keeps a small amount of residual cache in full precision
KVQuant [20]	Non-Uniform Quantization (NUQ)	Key: Per-channel (Pre-RoPE); Value: Per-token	Isolates the top 1% of outliers into a separate sparse fp16 representation
MiniCache [21]	Cross-Layer KV Merge	NA	Keeps highly distinct unmergeable state pairs in full precision
PALU [19]	Hidden-dimension compression with Low-Rank Projection and Reconstruction	Per-token per head group latent vector	Assigns higher ranks to critical layers

Attention Sink-Aware Quantization : The first token is stored in full FP16 precision because LLM tends to allocate a large attention score to the first token, even when the initial token is semantically less important.

By combining these techniques, KVQuant achieves aggressive compression while preserving model fidelity, making ultra-long context inference feasible.

3.3 Hybrid Memory Solution

Hybrid memory solutions address the limitations of GPU memory by leveraging multi-tier storage architectures to manage the KV cache efficiently. As context lengths grow, storing the entire KV cache on GPU becomes infeasible due to memory constraints and bandwidth bottlenecks. Hybrid approaches mitigate these challenges by offloading portions of the cache to slower but larger memory tiers, such as CPU memory, disk, or specialized accelerators. Hybrid memory solution aims to balance memory availability and computational efficiency. The solutions discussed in this section are summarized in Table 4.

A widely adopted hybrid memory strategy begins with treating KV storage like paged virtual memory. PagedAttention [22] is an attention algorithm “inspired by the operating system’s (OS) solution to memory fragmentation and sharing: virtual memory with paging” [22]. It divides the KV cache into fixed-size units called KV blocks, analogous to memory pages. These blocks do not need to be stored next to each other in the physical GPU memory. A block table maps logical blocks into physical locations, similar to page tables in OS. During attention computation, the model retrieves only the required blocks, operating block-by-block rather than assuming contiguous storage. Such block-level management enables efficient memory sharing, particularly in complex decoding scenarios such as parallel sampling, where multiple sequences share the same input prompt, and the KV blocks corresponding to the prompt are physically shared. To maintain consistency when a shared block is modified, PagedAttention employs a copy-on-write mechanism by duplicating only the affected block instead of the entire cache. This approach significantly reduces memory overhead while supporting scalable multi-sequence inference.

Another hybrid memory solution predicts future KV needs during inference. InfiniGen [23] introduces a dynamic KV cache management strategy that stores the cache in CPU memory and selectively transfers only the most critical KV pairs to the GPU for attention computation, minimizing data transfer overhead. Its key innovation lies in predictive prefetching, that is anticipating which tokens will be needed before they are actually required, enabling efficient prefetching.

To achieve this, InfiniGen leverages the high similarity between adjacent query vectors to calculate an approximate query vector, known as Partial Q. This approximation process is a single matrix multiplication using current layer input and the next layer’s query weight $W_Q^{(layer+1)}$, which is pre-loaded. The accuracy of $\tilde{Q}$ is further improved by applying a low-rank transformation M learned offline via SVD.

$$\tilde{Q}^{(layer+1)} = X^{(layer)} \cdot M \cdot W_Q^{(layer+1)} \quad (6)$$

Using $\tilde{Q}$ and KV metadata, InfiniGen predicts which KVs could be critical to the next layer, and prefetch them from CPU to GPU while the GPU is still processing the current layer. This overlapping of computation and data transfer significantly improves throughput for long-context inference.

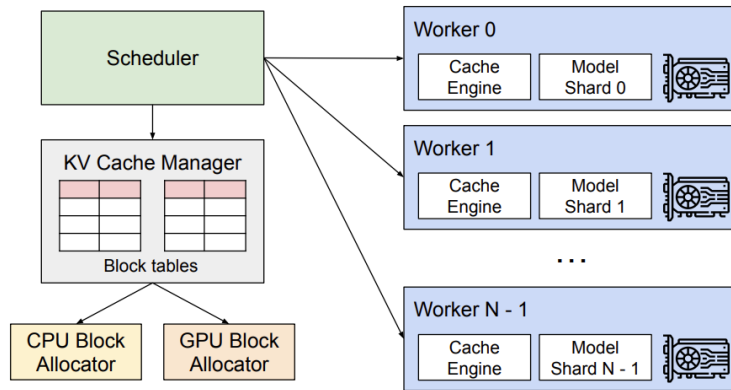


Figure 10: vLLM system overview [22].

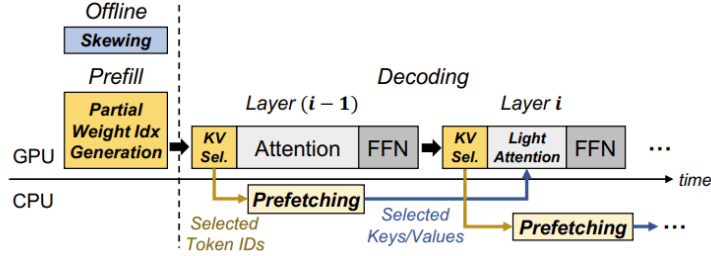


Figure 11: Operation flow of the prefetching module of InfiniGen. [23].

A layer-wise KV cache management strategy is proposed in LayerKV [24]. The core concept is to split KV cache by layers, keeping only a subset of layers on the GPU during the prefill stage while offloading some layers to CPU memory to reduce Time to First Token (TTFT). Prefill time refers to the time for the GPU to compute the first token. Offload time is the time required for the CPU to transfer KV data back to the GPU. To hide transfer latency, LayerKV selects the minimum number of layers to keep on the GPU such that: offload time $\leq$ prefill time. For example, in an 8-layer model, LayerKV may keep layers 1, 3, 5, and 7 on the GPU while offloading layers 0, 2, 4, and 6 to CPU memory. As the GPU processes layer 1, the CPU simultaneously transfers KV data for layer 0 back to the GPU, creating overlap between computation and communication. This overlap significantly reduces TTFT. Additionally, LayerKV integrates an SLO-aware scheduler that determines how many new inference requests can be initiated without violating Time Per Output Token (TPOT) guarantees. This ensures that both latency and throughput targets are met under multi-request serving scenarios.

A hardware–software co-design is explored in INF2 [25]. INF2 stands for “INference-INfinity”. This method leverages Computational Storage Devices (CSDs), which are special SSDs with attached accelerators like FPGA. The key mechanism is to offload attention computation to accelerators near storage. During the decoding phase, instead of transferring large KV caches back and forth between GPU and storage, INF2 stores the KV cache directly on the SSDs within the CSDs. The KV data is loaded into the attached accelerator via a private PCIe switch, allowing multi-head attention to be computed locally on the accelerator, close to where the KV resides. While the accelerator performs the attention computation near storage, the GPU concurrently processes the remaining computation like MLP. Newly generated KV entries are temporarily stored in RAM and will be sent to SSD in batches. By relocating attention computation to near-storage accelerators and reducing PCIe traffic, INF2 significantly enhances LLM throughput for long-context inference and alleviates pressure on GPU memory bandwidth.

Another system aims to reduce GPU idle time. The technique in KVPR [26] overlaps partial KV recomputation on GPU with data transfer from CPU to GPU, synchronizing both to minimize stalls. Instead of waiting for the full KV cache to arrive, KVPR first transfers a small portion of the input activations, enabling the GPU to immediately begin recomputing a corresponding subset of the KV cache. Meanwhile, the system continues to transfer the rest of the KV cache in parallel.

In a profiling stage, the system measures key parameters such as PCIe bandwidth, GPU compute throughput, and model sizes. Subsequently, the system determines whether the deployment target prioritizes low latency or high throughput. Based on this objective, a scheduling module selects between two processing strategies: row-wise scheduling for latency-sensitive workloads, or column-wise scheduling for throughput-oriented workloads. Using the profiling results, KVPR computes the optimal amount of KV cache to recompute. This calculation is performed using the parameters collected from the prefilling phase, where the small amount of KV cache’s recomputation time matches the transfer time for the remaining KV cache. This creates a tightly synchronized pipeline where GPU computation and data transfer overlap almost perfectly. During inference, the system first sends the activations needed for recomputation. The GPU begins partial KV regeneration as soon as they arrive, while the CPU continues streaming the remaining KV cache. Once both the recomputing and transferring phases finish, the recomputed and transferred KV cache segments are merged for the subsequent attention calculation.

A multi-tenant serving strategy is demonstrated in Oneiros [27], which temporarily remaps model parameters off GPU to free memory for KV cache expansion during heavy decoding workloads. It is designed for multi-tenant LLM serving on modern hardware with high bandwidth. It reuses GPU memory allocated for model parameters as extra space for KV cache, a process called “parameter remapping”. To be more specific, when the KV cache approaches the GPU’s memory limit, Oneiros triggers remapping to free space.

Table 4: Hybrid Memory Solutions Comparison Table

Method	Offload destination	Mechanism	Key optimization
Paged Attention [22]	CPU Memory (DRAM)	Paging the KV cache, inspired by virtual memory	Reducing memory fragmentation and enabling cache sharing
InfiniGen [23]	CPU Memory (DRAM)	Attention speculation and prefetching	Reducing PCIe transfer volume by only fetching essential data
LayerKV [24]	CPU Memory (DRAM)	Layer-wise management and SLO-aware scheduler to balance GPU/CPU residency	Reducing TTFT by minimizing queuing delays
INF2 [25]	Host Memory + NVMe SSDs (CSDs)	Attention-Near Storage (ANS) via Computational Storage Devices (CSDs)	High throughput for long-context models by leveraging internal storage bandwidth
KVPR [26]	CPU Memory (DRAM)	Overlapping partial cache recomputation on the GPU with asynchronous transfer	Minimizing GPU idle time caused by limited PCIe bandwidth
Oneiros [27]	CPU Memory (DRAM)	Parameter remapping to repurpose GPU memory from inactive models into KV cache space	Optimizing multi-tenant serving by reclaiming memory from idle models
CLO [28]	CPU Memory (DRAM)	Algorithm-system co-design with head-wise approximate caching and zero-copy transfer engine	Eliminating CPU bottlenecks and fully utilizing PCIe bandwidth

The system first identifies which parameters to offload for remapping. Normally, inactive models are the preferred candidates as their parameters are unlikely to be needed immediately. If all models are active, then the system will offload a subset of layers from each model in an evenly spaced manner. This strategy takes advantage of the sequential layer-by-layer execution inherent in autoregressive decoding. Since LLM inference repeatedly cycles through layers for each token, distributing remapped layers evenly ensures sufficient compute time between remapped layers, allowing parameter transfers from CPU to GPU to be effectively hidden behind ongoing computation. After offloading the inactive models or layers, the newly released GPU memory becomes available to store additional KV cache entries. If a remapped layer is later required, its parameters are fetched back from CPU to GPU memory, while the GPU continues computing the next available layers, overlapping communication with computation to minimize latency impact. When KV cache pressure subsides, Oneiros restores GPU memory to model parameters, reversing the remapping process as needed.

To reduce redundant KV transfers, Jiawei et al. proposed CLO [28]. It is a KV-cache offloading and reuse mechanism designed to accelerate LLM inference by intelligently moving KV cache between CPU and GPU memory. Its key insight is that Query vectors generated during adjacent decoding steps (generating one token after another) are often highly similar. This high temporal locality suggests that the corresponding KV entries needed for attention are also likely to be the same. CLO measures the cosine similarity between the current query and the previous one. If the similarity exceeds a threshold, the system assumes a high likelihood of accessing the same KV vectors and therefore reuses the previously loaded KV cache instead of fetching new KV data from CPU memory. This significantly reduces cache movement overhead. When similarity is low, CLO must fetch the correct KV cache to maintain accuracy. In this case, it uses a prefetching mechanism, inspired by InfiniGen, to predict KV requirements for the next layer and retrieves the data early. The offloading and fetching process relies on both a zero-copy transfer engine, which is built on GDRCopy developed by Nvidia, and a GPU-centric synchronization method to fully utilize the PCIe bandwidth and eliminate GPU stalls. Similar to other methods, CLO also incorporates outlier handling mechanisms. Certain attention heads have disproportionate influence on model output and are therefore designated as critical. CLO keeps the KV cache of these heads permanently in GPU memory to prevent latency spikes and accuracy degradation. For all other heads, the system follows the similarity-guided workflow, either reusing previous KV data or fetching new KV entries with prefetching as needed.

3.4 New Attention Calculation

The Transformer architecture, introduced in Attention Is All You Need [6], relies on Scaled Dot-Product Attention as its core mechanism. This method computes a compatibility score between each query and key vector pair to estimate how well they match. It scales the score by the square root of the hidden dimension, and applies a softmax function to

obtain normalized attention weights. These weights are then used to compute a weighted sum of the corresponding value vectors, producing the output representation.

While this approach has become the foundation of modern LLMs, its computational complexity of $O(n^2)$ with respect to sequence length poses significant challenges for long-context inference. Consequently, recent research has focused on developing alternative attention mechanisms that reduce complexity while preserving accuracy. These innovations include linear attention, log-linear attention, and hybrid methods that balance efficiency and expressiveness. We summarized the attention variants in Table 5.

One foundational line of work seeks to reduce the quadratic cost of softmax attention. Transformers-are-RNNs [29] is one of the first works that introduces the concept of linear attention for autoregressive transformers, demonstrating how the quadratic $O(N^2)$ complexity of softmax attention can be reduced to linear time $O(N)$. The key idea is to replace the softmax similarity function with a kernel-based linear attention. For a single query vector q (one token), the typical linear-attention output is written as:

$$V'_i = \frac{\phi(Q_i)^T \sum_{j=1}^i \phi(K_j) V_j^T}{\phi(Q_i)^T \sum_{j=1}^i \phi(K_j)}$$

It can be simplified into:

$$V'_i = \frac{\phi(Q_i)^T S_i}{\phi(Q_i)^T Z_i}$$

where

$$S_i = \sum_{j=1}^i \phi(K_j) V_j^T$$

$$Z_i = \sum_{j=1}^i \phi(K_j)$$

Here, $\phi(\cdot)$ is a feature map. It is a function that is applied to each query or key vector elementwise that produces a new vector of the same size. It can turn the original similarity (q, k) from: similarity $(q, k) = \text{softmax}(q^T k)$ into an approximation of the form similarity $(q, k) \approx \phi(q) \cdot \phi(k)$ that can be written as a dot product of transformed vectors. By expressing similarity as a dot product between transformed vectors, the attention computation can be rearranged into a sequence of associative matrix multiplications. This allows the key-value accumulation term S to be computed incrementally, which effectively transforms the attention mechanism into a recurrence that operates in linear time and memory with respect to sequence length.

A middle-ground alternative is explored in Log-Linear Attention [30], allowing the cost to be between the traditional softmax attention (which has quadratic $O(N^2)$ complexity) and efficient linear-attention variants. Its central idea is to hierarchically summarize past tokens so that attention computation scales in $O(N \log N)$ time and $O(\log N)$ memory.

The method organizes token history using a Fenwick-tree-style structure, where past tokens are grouped into buckets whose sizes are powers of two. Each token gets assigned to a bucket level based on its position in the sequence. Each new token is first placed in the finest-grained bucket, while older tokens progressively “age” into coarser buckets. For example, when there are 8 tokens, $Bucket_0$ handles the current token, $Bucket_1$ holds the last 1 token, $Bucket_2$ holds the last 2 tokens, and $Bucket_3$ holds the last 4 tokens. Therefore, at time step t , there will be $\log_2(t)$ buckets, each representing a summary over a different time scale. As the sequence grows, buckets automatically merge following the Fenwick-tree update logic, ensuring that at most $\log_2(t)$ buckets are active. This hierarchical nature results in a computational cost of $O(N \log N)$ (log-linear time) and memory cost of $O(\log N)$ (logarithmic space). For each bucket ℓ at time t , the system computes a summary matrix $S_t^{(\ell)}$. Then, each bucket gets a weight and uses it to compute the output. This process is represented in the following formula:

$$o_t = \sum_{\ell=0}^{L-1} \lambda_t^{(\ell)} q_t^T \left(\sum_{s \in B_t^{(\ell)}} v_s k_s^T \right) = \sum_{\ell=0}^{L-1} \lambda_t^{(\ell)} q_t^T S_t^{(\ell)}$$

A regression-inspired formulation appears in Local Linear Attention (LLA) [31], where attention is approximated locally, striking a balance between softmax responsiveness and linear-attention efficiency. Local Linear Attention is inspired by the similarity between attention and regression. Conceptually, attention takes past keys and values as “training examples” and produces an output for the current query, similar to how regression predicts a new data point from observations. Under this interpretation, Softmax Attention behaves like local constant regression because it looks

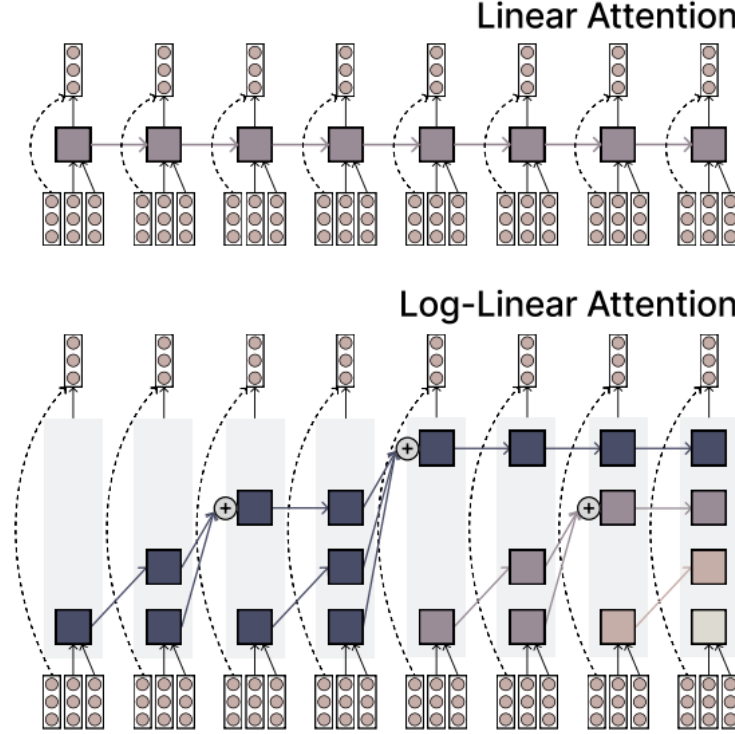


Figure 12: Standard linear attention (top) vs. loglinear attention (bottom). The input consists of query, key, and value vectors [30].

at nearby keys and averages their value; while Linear Attention is like global linear regression because it fits a global straight line for all data. Based on such observation, the authors proposed Local Linear Attention, which is similar to local linear regression. This enables LLA to adapt more effectively to non-stationary or time-varying patterns in the sequence, offering a middle ground between locally responsive softmax attention and the globally smoothed behavior of linear attention. In practice, LLA fits a small local linear model around each query by selecting a neighborhood of relevant key-value pairs and estimating a local linear approximation. The output is then computed based on this locally fitted model, allowing the attention mechanism to respond dynamically to changes in the underlying data distribution while retaining the efficiency benefits of linear-attention structures.

A more expressive linear framework is introduced in KIMI Linear [32], combining dynamic gating mechanisms with a hybrid of linear and softmax layers to improve representational power.

KIMI Linear introduces Kimi Delta Attention (KDA), an expressive form of linear attention designed to improve the representational capacity of linear-time architectures. KDA is the central innovation of the work, and KIMI Linear integrates KDA and full softmax attention layers in a 3:1 hybrid ratio, which the authors identify as the optimal balance between efficiency and model accuracy.

KDA incorporates two dynamic gating mechanisms to control how memory evolves over time. They are forget gate (α) and update rate (β). Forget gate (α) is a decay factor that determines how much of the previous memory state S_{t-1} is retained when updating the memory at time step t . If α is close to 1, the memory state keeps most of the old info; otherwise, if α is close to 0, it forgets quickly. Update rate (β) determines how strongly the new KV pair influences the memory update at time step t . Large β value means new info dominates, whereas small β value means new info has little effect. Both α and β are calculated from the input token using small neural networks, whose parameters are learned during model training. In KDA, the memory state is represented by matrix S . It is calculated using the following formula:

$$S_t = (I - \beta_t k_t k_t^T) \text{Diag}(\alpha_t) S_{t-1} + \beta_t k_t v_t^T \in \mathbb{R}^{d_k \times d_v}$$

Where: S_{t-1} is the previous memory state; $\text{Diag}(\alpha_t)$ is a diagonal matrix where each diagonal entry is one element of α_t ; $(I - \beta_t k_t k_t^T)$ adjusts memory so it doesn't overfit old associations; $\beta_t k_t v_t^T$ store the new key-value pair in memory.

Table 5: Attention Variants – Mechanisms, Complexities, and Features

Method	Mechanism	Training Time Complexity	Decoding Time Complexity per step	Decoding Space Complexity (Total Memory)	Features
Softmax	Scaled Dot-Product Attention & Multi-Head Attention	$O(T^2)$	$O(T)$	$O(T)$	High expressivity, high cost
Linear	Replaces softmax with a linear dot-product of kernel feature maps	$O(T)$	$O(1)$	$O(1)$	Limited expressivity and low cost
Log Linear	Replaces the fixed-size hidden state with a logarithmically growing set of hidden states	$O(T \log T)$	$O(\log T)$	$O(\log T)$	Balances efficiency and expressiveness
Local Linear	Use Local Linear Regression to perform query-specific local linear fitting	$O(T^2)$	$\sim O(T)$	$O(T)$	Superior bias–variance trade-off; high cost
KIMI Linear	Hybrid architecture (KDA + Multi-Head Latent Attention)	Mainly $O(T)$ due to KDA	$O(1)$	$O(1)$	Outperforms full attention, but requires hybridization (e.g., 3:1 ratio with full attention) to maintain global information flow

The dynamic memory state allows the model to retain specific information components selectively. And with S , the output can be calculated with:

$$o_t = S_t^T q_t \in \mathbb{R}^{d_v}$$

Beyond KDA, KIMI Linear incorporates additional optimizations. Instead of updating one token at a time, Kimi Linear processes tokens in chunks to increase computational parallelism and improve throughput. Also, KIMI Linear combines KDA (linear attention) and full attention at 3:1 ratio, which the authors empirically find delivers the best trade-off between efficiency and performance, preserving accuracy while significantly reducing memory and compute cost.

3.5 Combination Methods

Combination methods integrate multiple optimization strategies to achieve greater efficiency in KV cache management than any single technique alone. These approaches often combine eviction, compression, and hybrid memory solutions to balance memory savings, computational cost, and accuracy. For example, some methods apply token selection algorithms alongside quantization to reduce cache size, whereas others combine compression with offloading to CPU or disk to handle extremely long contexts. By leveraging complementary techniques, combination strategies aim to overcome the limitations of individual methods and deliver scalable solutions for large-context inference.

FlexGen[33] combines compression and offloading for extreme resource constraints. It enables inference for massive LLMs using only one single GPU with limited memory. This is done by splitting model weights, activations, and KV cache across the GPU, CPU memory, and disk. To determine the optimal placement and movement of these components, FlexGen constructs a cost model that estimates latency, bandwidth, and memory constraints for each device. It then solves a linear programming optimization problem to minimize the per-token generation time, yielding an execution plan that balances compute and I/O overhead. To further reduce memory footprint, FlexGen applies 4-bit compression to both model weights and the KV cache using group-wise quantization, where each small group of values shares a common scaling range (min/max). This achieves substantial compression with minimal accuracy degradation. It needs to be noted that FlexGen is designed specifically for high-throughput workloads, such as batched prompt processing, rather than low-latency, single-request scenarios. To sustain large batch sizes on limited GPU memory, it uses a zig-zag block scheduling strategy, which means processing multiple batches across one computational layer before moving to the next layer, instead of processing an entire batch through the full model sequentially. This strategy maximizes the reuse of model weights already loaded onto the fast GPU memory, thus reducing repeated loading and minimizing slow I/O transfers.

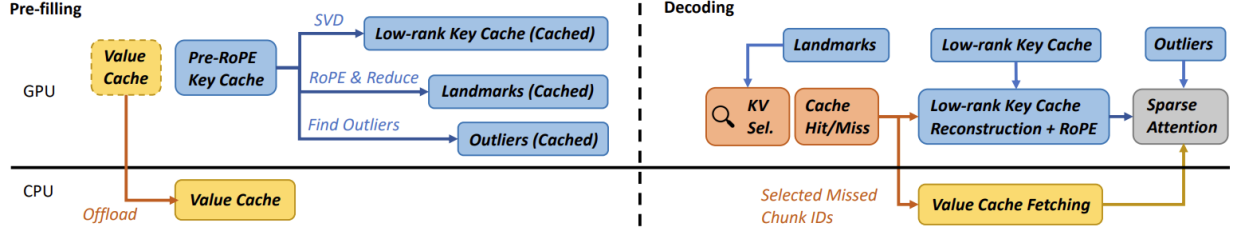


Figure 13: During Pre-filling, ShadowKV offloads the value cache to the CPU while maintaining a low-rank key cache, landmarks, and outliers on the GPU. During decoding, it employs landmarks for sparse attention. [35].

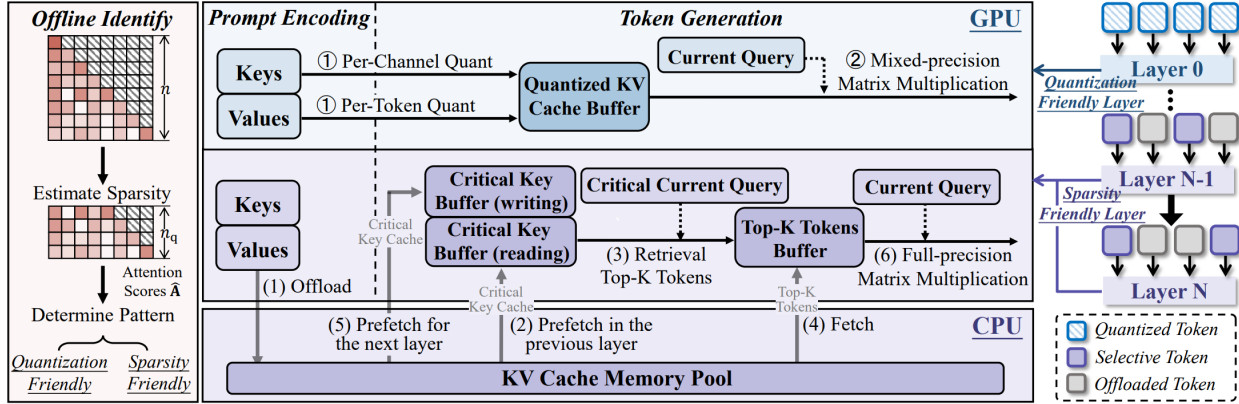


Figure 14: System overview of TailorKV. Offline identification categorizes the layers into quantization-friendly and sparsity-friendly. For quantization-friendly layers, we employ aggressive static quantization. For sparsity-friendly layers, we dynamically retrieve Top-K tokens. Critical current query and critical key cache represent the outliers in the query and key cache, respectively. [36].

A sparsity-aware quantized design is presented in Q-Hitter [34], where tokens are selected based on both attention importance and quantization robustness. In other words, the token-selection mechanism identifies tokens that are both semantically important and amenable to quantization, enabling efficient storage of a sparse, quantized KV cache. For each new token, the system computes two metrics based on its key and value vectors: the attention score and quantization error. Attention score reflects the token’s importance for future predictions by measuring how strongly it is likely to influence subsequent queries. Quantization error shows the amount of information lost if the token’s KV pair were stored in a low-bit representation. These two measurements are combined, via a balancing mechanism introduced in the paper, into a unified score S . Tokens with high S values are both important and robust to quantization noise. Therefore, Q-Hitter selects the top- K tokens according to S and stores only their KV entries in quantized form. Tokens with lower scores are discarded, resulting in a sparse KV cache that preserves critical information while significantly reducing memory footprint and I/O cost.

ShadowKV [35] is a hybrid approach that adopts both cache compression and hybrid memory. It stores compressed keys on GPU while offloads values to CPU memory. The key insight is that pre-RoPE keys exhibit strong low-rank structure, enabling effective compression without accuracy degradation. During the prefill phase, ShadowKV applies Singular Value Decomposition (SVD) to the pre-RoPE key matrix, retains only the top-rank components, and stores these compressed keys in GPU memory. In contrast, value vectors do not exhibit low-rank properties and therefore cannot be compressed effectively. To reduce GPU memory usage, ShadowKV offloads values to CPU memory. To enable efficient selective retrieval during decoding, the system divides post-RoPE keys into chunks and computes a landmark vector (typically the mean) for each chunk. It then compares each token’s post-RoPE key with its corresponding landmark. The tokens, whose keys showing low cosine similarity with the landmark, are marked as outliers because they cannot be well-represented by the chunk summary. These outlier tokens are fully stored on the GPU. During the decoding phase, ShadowKV uses the landmark vectors to estimate which chunks are likely to contribute most to the current attention computation. Only the values corresponding to these important chunks are fetched from CPU memory. Meanwhile, the compressed pre-RoPE keys stored on GPU are decompressed to reconstruct the necessary key vectors. Once both

the reconstructed keys and the fetched values are available, the system performs the standard attention computation to generate the next token.

Layer-specific strategies are combined in TailorKV[36]. TailorKV as shown in Fig. 14 is a hybrid KV-cache optimization framework that exploits a key empirical observation about transformer layers. That is, in transformer models, shallow layers (closer to the input) tend to distribute their attention broadly across the sequence and are therefore more amenable to quantization; while deeper layers focus attention on a small subset of critical tokens, making them better suited for sparsity-based offloading. TailorKV combines these complementary strategies to support efficient long-context inference. During the prefill phase, TailorKV analyzes each layer’s attention distribution. It computes attention scores between the last query against all keys in that layer. Then it selects the top-k attention scores and sums them. The sum of these top-k values reflects how concentrated the layer’s attention is. A small sum indicates that this layer’s attention is spread out, suggesting that the layer is quantization-friendly. A large sum indicates that the attention is concentrated on a few tokens, meaning the layer is sparsity-friendly. Based on this metric, TailorKV assigns each layer to one of two regimes. For quantization-friendly layers, the KV cache will be compressed aggressively and stored compactly on GPU, reducing memory footprint while preserving accuracy due to their inherent global focus. For the sparsity-friendly layers, their KV caches are offloaded to CPU memory during prefill phase. Among them, only the most critical tokens (identified dynamically) will be fetched to GPU during decoding. To ensure high throughput, TailorKV uses double buffering, overlapping CPU-GPU data transfers with GPU computation, so that fetching and attention updates proceed concurrently.

4 Comparative Analysis

Table 6: Comparison of KV Cache Optimization Techniques

Technique	Memory	Speedups	Accuracy loss	Tradeoffs
H ₂ O [1]	Up to 5–10× memory reduction	Up to 29× throughput improvement; $\leq 1.9\times$ lower latency compare to FlexGen	Comparable to baseline	accumulated-attention bias; risk of heavy-hitter loss
SnapKV [2]	8.2× memory efficiency	3.6× generation speedup	Comparable to baseline	Does not optimize prefill; cannot extend inherent model context limits
NACL [9]	Up to 5× KV reduction	O(1) single-shot eviction; major prefill simplification	95% performance retention	Proxy-token selection heuristic; limited ultra-long behavior
InfiniPot [10]	Tested with 4k to 1M tokens	“Performance regardless of context length”	Consistent performance regardless of context length and optimizing memory use	Fixed compression ratio may not be optimal for all data types
HASHEVICT [11]	30–70% compression	1.5–2× prefill speedup against baseline methods such as H ₂ O, Scissorshands, and 17× prefill/2× decoding speed against FastGen	Decent performance except at very small budgets (e.g., 10% capacity)	Irreversible eviction; quality drops sharply at 10% cache budget
MorphKV [13]	Up to 5× over Full-Attention.	Up to 4.68× faster than SnapKV	Comparable to SnapKV	Performance is sensitive to hyperparameters like window size and fusion functions
RocketKV [14]	Up to 400× compression	Up to 3.7× speedup	Negligible accuracy loss	Focused on decode; limited prefill optimization
KVzip [15]	Up to 70% eviction	2× reduction in FlashAttention decoding latency	Negligible accuracy loss	Reconstruction overhead (amortizable over multiple queries); no formal guarantees on information loss
Ada-KV [3]	4× cache reduction	Comparable decoding latency to SnapKV	Slight quality improvement ($\sim 5\%$)	Allocation only within layers, not across model

Continued on next page

Technique	Memory	Speedups	Accuracy loss	Tradeoffs
KIVI [5]	2.6× peak memory reduction	2.35×–3.47× throughput	<2% accuracy drop for most models	Models with one KV head may require 4-bit quantization to maintain accuracy; quantization overhead due to initial cost for the quantization
MiniCache [21]	Up to 41% memory reduction	~5× throughput	Minimal loss	Limited to merging only two layers at a time; restricting higher compression
PALU [19]	~50% KV compression	Up to 1.89× (RoPE) or 2.91× (with quantization)	Comparable to baseline	Reconstruction overhead for keys (especially with RoPE)
KVQuant [20]	3.7–6.9× memory savings	Up to ~1.7× speedup	<0.1 perplexity degradation with 3-bit quantization	Long-context training challenges; more complex de-quantization process
PagedAttention [22]	Offload based (no reduction)	2–4× higher throughput with the same latency	Lossless	Kernel overhead; computational overhead for managing logical to physical mapping
InfiniGen [23]	Offload based (no reduction)	1.63–32.9× speedup	Comparable accuracy with >15% Relative KV Cache Size	Requires additional GPU memory to store partial weights used for speculation; Specifically designed for CPU-offloading systems
LayerKV [24]	Offload based (no reduction)	Up to 69× TtFT improvement	Lossless	Slight throughput trade-off under high-load decoding
INF2 [25]	Offload based (no reduction)	3.46× throughput; KV I/O overhead reduced by >80%	Lossless	CPU coordination overhead; Requires CSD hardware
KVPR [26]	Offload based (no reduction)	Up to 35.8% lower latency, 46.2% higher throughput compared to DeepSpeed and HuggingFace Accelerate	Lossless	Decoding focus only; limited to single-GPU or data-parallel setups
Oneiros [27]	Offload based (no reduction)	44.8%–82.5% time-between-token latency reduction; 20.7%–99.3% TtFT reduction; 6.6%–86.7% throughput improvement compared to vLLM	Lossless	Requires high bandwidth for CPU–GPU
CLO [28]	Offload based (no reduction)	9.3%–66.6% throughput improvement compared to SOTA baseline (RetroInfer and InfiniGen)	Near lossless: ≤ 0.42 accuracy drop compared to original models	Approximation leads to sacrificing a small amount of cache hit ratio to gain speed; Hyperparameter tuning required manual tuning; PCIe 4.0 dependency
LinearAttention [29]	$O(1)$ memory	Up to 4000× faster on long sequences compared to vanilla transformers	Strong on synthetic tasks; weaker on complex tasks	Performance sensitive to choice of feature map kernels; accuracy gap for reasoning
Log Linear Attention [30]	$O(\log T)$ memory	3× speedup over naive implementations	Better than linear; below full attention	Performance gap compared to full attention; engineering complexity
Local Linear Attention [31]	Similar to softmax	Faster than softmax	Outperforms Softmax Attention and Linear Attention on associative and regression tasks	Trades off speedups for accuracy; limited memory benefit
KIMI [32]	Up to 75% KV reduction	Up to 6× throughput at 1M context	Outperforms full attention	Kernel dependency for maximum efficiency

Continued on next page

Technique	Memory	Speedups	Accuracy loss	Tradeoffs
FlexGen [33]	Up to $10\times$ memory reduction	$40\times$ to $100\times$ higher maximum throughput compared to DeepSpeed Zero-Inference and HuggingFace Accelerate	Negligible accuracy loss with 4-bit	High latency; PCIe/Disk bandwidth bottleneck for KV retrieval; insufficient for small batches
Q-Hitter [34]	Up to $20\times$ memory reduction	Up to $33\times$ vs HF Accelerate	Full quality preservation	Computational overhead for calculating quantization errors and performing dequantization
ShadowKV [35]	$6\times$ GPU memory reduction	$3.04\times$ throughput improvement	High accuracy until sparse budget $< 1.56\%$	Performance partially dependent on PCIe bandwidth
TailorKV [36]	$\sim 73.8\%$ GPU memory reduction	$8\text{--}18\times$ faster than standard offloading	Near lossless	Prefill bottleneck; system complexity

5 Scenarios

5.1 Long Context ($>1\text{M}$ tokens) for Single Request:

Cache eviction and cache compression are generally well-suited for handling long-context workloads involving single requests. In such settings, the primary bottleneck arises from the substantial memory footprint associated with processing extended sequences. Because cache eviction and cache compression are explicitly designed to substantially reduce the KV cache memory requirements, while incurring only negligible accuracy degradation, they represent the most appropriate optimization strategies. Hybrid techniques that incorporate these two methods are likewise suitable for this scenario.

Beyond these categories, KIMI from category 4 is also applicable, as its authors explicitly note that Kimi Linear is designed to process million-token sequences while achieving up to $6.3\times$ higher throughput and reducing memory usage by 75% compared to standard models.

5.2 Minimal model modification:

Cache eviction and cache compression are generally recommended for minimal model modification, as both approaches directly target reductions in memory footprint. Among these methods, Ada-KV, SnapKV, and KIVI are particularly suitable. These techniques are fine-tuning-free and are designed for seamless “plug-and-play” integration into existing models with minimal architectural adjustments. If compression or quantization is preferred over eviction, KIVI represents the most appropriate choice, as it offers a tuning-free 2-bit solution capable of handling outliers without necessitating model retraining.

Linear, log-linear, local linear, and KIMI methods are generally not recommended for scenarios requiring minimal model modification. These methods involve fundamental architectural changes, new attention mechanisms, or extensive training, which contrast with the “plug-and-play” or “training-free” nature of post-training KV cache management strategies.

5.3 High throughput serving:

Hybrid memory solutions are generally well-suited for achieving high throughput. Among the methods in this category, PagedAttention (vLLM) is the most appropriate, as it is specifically designed to maximize throughput by enabling a larger number of requests to be processed within a single batch. Similarly, Oneiros targets multi-tenant LLM serving and attains up to 86.7% higher throughput than vLLM by remapping parameter memory for KV-cache usage.

ShadowKV is also a strong candidate, as it integrates cache compression with hybrid memory techniques. By offloading values and employing low-rank keys, ShadowKV supports batch sizes up to $6\times$ larger and improves throughput by up to $3.04\times$.

For scenarios requiring extreme throughput in resource-constrained environments (e.g., a single GPU), FlexGen and its successor Q-Hitter utilize aggressive offloading strategies. These systems achieve up to $33\times$ higher throughput than standard offloading frameworks through the co-design of quantization and sparsity.

5.4 Edge/memory-limited devices

For edge or memory-limited devices, cache eviction and cache compression are also effective techniques, as they reduce memory pressure by minimizing the overall memory footprint. Among the methods in this category, InfiniPot is explicitly designed for on-device environments (mobile/edge) where VRAM or NPU memory constitutes a strict constraint; it employs Continual Context Distillation to maintain the KV cache within a fixed-size “pot.” TailorKV likewise targets resource-limited GPUs and was specifically developed to serve 128k-context 8B models on a single RTX 3090 GPU.

Conversely, methods with demanding resource requirements are not preferred due to the inherently constrained nature of edge environments. Most hybrid-memory approaches fall into this scenario. For example, PagedAttention is optimized for high-end GPUs, while Oneiros depends heavily on extremely high CPU–GPU bandwidth (450–900 GB/s), which is rarely available on edge devices. These methods are therefore unsuitable for edge deployment due to their hardware requirements.

5.5 Multi-turn Conversations:

Standard cache-eviction methods such as H₂O are not suitable for multi-turn conversations, as they permanently discard tokens that may be required in later turns. However, several methods are specifically optimized for such dialogue-oriented scenarios. For example, RocketKV-MT is a variant of the standard RocketKV designed for multi-turn settings; it retains all KV tokens in memory for future turns while constraining token selection in the current turn. KVzip is a query-agnostic method that optimizes reusable compressed KV caches, enabling efficient inference across diverse future queries within a dialogue. The authors note that its “overhead can be amortized over multiple queries.” ShadowKV is another strong option, as it demonstrates “multi-turn capability” by sharing low-rank subspaces between a sequence and its continuation, thereby maintaining accuracy across multiple rounds of interaction.

Besides standard cache eviction, methods that impose substantial latency are also poorly suited for this setting, as multi-turn conversations also require low latency and responses must be generated quickly to maintain an interactive user experience. For example, FlexGen is optimized for latency-insensitive batch processing, and its inherent delays render it impractical for interactive dialogue.

5.6 Prefill-Heavy Workloads:

NACL is recommended because it performs efficient eviction in a single operation during the encoding phase, rather than step-by-step during decoding. This design reduces time complexity and minimizes the computational overhead typically associated with managing large-scale KV caches.

HASHEVICT is also a strong candidate, as it achieves a 1.5×–2× prefill speedup by making eviction decisions prior to the attention computation.

If the primary bottleneck is Time to First Token (TTFT) for very long prompts, LayerKV (69× improvement in TTFT) or MiniCache (significant prefill efficiency) are recommended. In the LayerKV paper, the authors state that “Comprehensive evaluations on representative models, ranging from 7B to 70B parameters, across various GPU configurations, demonstrate that LayerKV improves TTFT latency up to 69×.” [24]

Lastly, CLO processes the full prompt in parallel during the prefill phase and employs speculative sparse prefetching to hide subsequent loading overhead, making it a suitable option as well.

5.7 Accuracy-Critical Reasoning:

Hybrid memory solutions such as PagedAttention are preferred for accuracy-critical reasoning tasks. Methods in this category improve overall efficiency by leveraging hybrid memory infrastructures rather than modifying data representations or altering model components. As a result, they maintain lossless accuracy, making them suitable for scenarios where reasoning precision is essential.

Conversely, cache-eviction and compression-based techniques reduce memory footprint at the cost of accuracy. Likewise, linear attention and log-linear attention methods tend to underperform in highly complex reasoning scenarios when compared to full attention. Therefore, these approaches should be avoided in accuracy-critical applications.

5.8 Hardware-Specific Constraints:

For configurations with high PCIe bandwidth (e.g., NVIDIA GH200 systems), Oneiros and CLO are superior choices because they leverage the interconnect to offload KV caches with negligible overhead. Oneiros exploits the high CPU–GPU bandwidth available on specialized hardware such as the NVIDIA Grace Hopper (GH200) to repurpose parameter memory for KV-cache storage, while CLO utilizes zero-copy transfer engines and custom CUDA kernels to fully exploit PCIe bandwidth on modern GPU platforms. Therefore, both methods are suitable for this scenario.

INF2 is designed for fast inference using Computational SSDs (CSDs) equipped with ASICs or FPGAs to offload KV-related operations. When such hardware is available, this method should be considered the top choice.

FlexGen is presented as a high-throughput generation engine capable of running LLMs with limited GPU memory. It can be flexibly configured under various hardware resource constraints by aggregating memory and computation across the GPU, CPU, and disk.

6 Summary

We categorized KV-cache optimization techniques into the following categories: cache eviction, cache compression, hybrid memory solutions, new attention mechanisms, and combination methods. This survey also reveals distinct strengths, limitations, and ideal deployment scenarios for each.

First, we observe that eviction, compression, and their combination approaches increasingly dominate ultra-long-context scenarios, particularly when context lengths exceed one million tokens. Methods such as RocketKV, KVzip, and ShadowKV achieve high compression ratios with minimal accuracy degradation by combining coarse-grained token selection with fine-grained reconstruction or low-rank approximations. These methods substantially reduce GPU memory pressure while retaining global contextual fidelity, making them well-suited for large-context, single-request workloads.

Second, hybrid memory solutions are essential for scaling beyond physical GPU limits through CPU offloading and efficient block-based memory allocation because they fundamentally change where KV resides rather than what is stored. This category of methods consistently excels under high-throughput or multi-tenant serving conditions. Systems like PagedAttention, Oneiros, KVPR, and INF2 demonstrate that offloading, remapping, and near-storage computation are more effective than data-centric optimizations when throughput is the primary metric. Their performance benefits are largely orthogonal to model structure, enabling lossless accuracy and scalable serving across diverse workloads. Therefore, hybrid methods are the strongest fit for data-center deployments requiring high concurrency, where accuracy remains lossless, and hardware interconnect speed can be fully exploited.

Alternative attention mechanisms, including linear, log-linear, local linear, and KIMI, offer asymptotically better scaling than standard softmax attention, reducing complexity from $O(N)$ to $O(N \log N)$ or $O(N)$. These methods are promising for architectural redesigns where ultra-long context is a first-class goal. However, they require full model retraining and still lag in accuracy-critical reasoning, making them less suitable as drop-in optimizations for existing LLMs. Their long-term significance lies in pointing to future transformer successors that natively support million-token sequences without relying on cache pruning.

Hybrid strategies such as FlexGen, Q-Hitter, ShadowKV, and TailorKV demonstrate that no single optimization dominates across all scenarios. By mixing sparsity, quantization, compression, and offloading, these frameworks deliver efficiency unattainable by individual techniques, particularly under extreme resource constraints. Combination methods are especially effective when workloads demand balanced latency, throughput, and memory savings, or when operating on consumer-grade GPUs with limited VRAM.

Overall, across the surveyed landscape of KV-cache optimization techniques, several clear patterns emerge. Eviction and compression strategies dominate for ultra-long-context workloads, while hybrid memory systems dominate for high-throughput serving. Standalone compression excels in bandwidth-bound environments, and new attention mechanisms provide a path toward fundamentally scalable architectures. Ultimately, the future of long-context inference lies in integrated, multi-stage KV optimization pipelines that adapt dynamically to context length, system load, and hardware constraints.

References

- [1] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, Z. Wang, and B. Chen, “H₂O: Heavy-hitter oracle for efficient generative inference of large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2306.14048>

- [2] Y. Li, Y. Huang, B. Yang, B. Venkitesh, A. Locatelli, H. Ye, T. Cai, P. Lewis, and D. Chen, “Snapkv: Llm knows what you are looking for before generation,” 2024. [Online]. Available: <https://arxiv.org/abs/2404.14469>
- [3] Y. Feng, J. Lv, Y. Cao, X. Xie, and S. K. Zhou, “Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference,” 2025. [Online]. Available: <https://arxiv.org/abs/2407.11550>
- [4] G. von Laszewski, W. Brewer, J. Thiagalingam, J. Papay, A. Foundjem, P. Luszczek, M. Emani, S. V. Moore, V. J. Reddi, M. D. Sinclair, S. Lobentanzer, S. Goswami, B. Hawks, M. Colombo, N. Tran, C. R. Kirkpatrick, A. Alsudais, G. Barrett, T. Li, K. Morehouse, S. Venkataraman, R. Jain, K. Mathur, V. Lu, T. Singh, K. Z. Mirza, K. Chen, S. Kunapuli, G. Farrell, R. Umeton, and G. C. Fox, “AI Benchmark Democratization and Carpentry,” *arXiv [cs.AI]:2512.11588*, pp. 1–43, Dec. 2025. [Online]. Available: <https://arxiv.org/abs/2512.11588>
- [5] Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, V. Braverman, Beidi Chen, and X. Hu, “Kivi : Plug-and-play 2bit kv cache quantization with streaming asymmetric quantization,” 2023. [Online]. Available: https://www.researchgate.net/profile/Zirui-Liu-29/publication/376831635_KIVI_Plug-and-play_2bit_KV_Cache_Quantization_with_Streaming_Asymmetric_Quantization/links/658b5d282468df72d3db3280/KIVI-Plug-and-play-2bit-KV-Cache-Quantization-with-Streaming-Asymmetric-Quantization.pdf
- [6] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” 2023. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [7] OpenAI, “Model comparison: GPT-5 and other openai models,” <https://platform.openai.com/docs/models/compare?model=gpt-5>, 2026, accessed: 2026-03-13.
- [8] Meta AI, “Llama 4: Multimodal intelligence,” <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>, 2026, accessed: 2026-03-13.
- [9] Y. Chen, G. Wang, J. Shang, S. Cui, Z. Zhang, T. Liu, S. Wang, Y. Sun, D. Yu, and H. Wu, “Nacl: A general and effective kv cache eviction framework for llms at inference time,” 2024. [Online]. Available: <https://arxiv.org/abs/2408.03675>
- [10] M. Kim, K. Shim, J. Choi, and S. Chang, “Infinipot: Infinite context processing on memory-constrained llms,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.01518>
- [11] M. Liu, T. Rabbani, T. O’Halloran, A. Sankaralingam, M.-A. Hartley, F. Huang, C. Fermüller, and Y. Aloimonos, “Hashevict: A pre-attention kv cache eviction strategy using locality-sensitive hashing,” 2025. [Online]. Available: <https://arxiv.org/abs/2412.16187>
- [12] L. Metcalf and W. Casey, *Cybersecurity and Applied Mathematics*. Cambridge, MA: Syngress, Elsevier, 2016.
- [13] R. Ghadia, A. Kumar, G. Jain, P. Nair, and P. Das, “Dialogue without limits: Constant-sized kv caches for extended responses in llms,” 2025. [Online]. Available: <https://arxiv.org/abs/2503.00979>
- [14] P. Behnam, Y. Fu, R. Zhao, P.-A. Tsai, Z. Yu, and A. Tumanov, “Rocketkv: Accelerating long-context llm inference via two-stage kv cache compression,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.14051>
- [15] J.-H. Kim, J. Kim, S. Kwon, J. W. Lee, S. Yun, and H. O. Song, “Kvzip: Query-agnostic kv cache compression with context reconstruction,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.23416>
- [16] NVIDIA. (2023) Nvidia tensorrt-llm supercharges large language model inference on nvidia h100 gpus. Accessed: 2026-03-13. [Online]. Available: <https://developer.nvidia.com/blog/nvidia-tensorrt-llm-supercharges-large-language-model-inference-on-nvidia-h100-gpus/>
- [17] T. Jeong, “4bit-quantization in vector-embedding for rag,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.10534>
- [18] D. Lee, S. Choi, and I. J. Chang, “Qrazor: Reliable and effortless 4-bit llm quantization by significant data razoring,” 2025. [Online]. Available: <https://arxiv.org/abs/2501.13331>
- [19] C.-C. Chang, W.-C. Lin, C.-Y. Lin, C.-Y. Chen, Y.-F. Hu, P.-S. Wang, N.-C. Huang, L. Ceze, M. S. Abdelfattah, and K.-C. Wu, “Palu: Compressing kv-cache with low-rank projection,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.21118>
- [20] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, “Kvquant: Towards 10 million context length llm inference with kv cache quantization,” 2025. [Online]. Available: <https://arxiv.org/abs/2401.18079>
- [21] A. Liu, J. Liu, Z. Pan, Y. He, G. Haffari, and B. Zhuang, “Minicache: Kv cache compression in depth dimension for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2405.14366>
- [22] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023. [Online]. Available: <https://arxiv.org/abs/2309.06180>

- [23] W. Lee, J. Lee, J. Seo, and J. Sim, “Infinigen: Efficient generative inference of large language models with dynamic kv cache management,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.19707>
- [24] Y. Xiong, H. Wu, C. Shao, Z. Wang, R. Zhang, Y. Guo, J. Zhao, K. Zhang, and Z. Pan, “Layerkv: Optimizing large language model serving with layer-wise kv cache management,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.00428>
- [25] H. Jang, S. Noh, C. Shin, J. Jung, J. Song, and J. Lee, “Inf²: High-throughput generative inference of large language models using near-storage processing,” *arXiv preprint arXiv:2502.09921*, 2025.
- [26] C. Jiang, L. Gao, H. E. Zarch, and M. Annavaram, “Kvpr: Efficient llm inference with i/o-aware kv cache partial recomputation,” 2025. [Online]. Available: <https://arxiv.org/abs/2411.17089>
- [27] R. Li, S. Pal, V. N. Pullu, P. Sinha, J. Ryoo, L. K. John, and N. J. Yadwadkar, “Oneiros: Kv cache optimization through parameter remapping for multi-tenant llm serving,” 2025. [Online]. Available: <https://arxiv.org/abs/2507.11507>
- [28] J. Yi, P. Gong, Y. Bai, J. Ruan, S. Wang, P. Wang, H. Wang, W. Wang, X. Zhu, F. Wu, and C. Li, “Clo: Efficient llm inference system with cpu-light kvcache offloading via algorithm-system co-design,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.14510>
- [29] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret, “Transformers are rnns: Fast autoregressive transformers with linear attention,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.16236>
- [30] H. Guo, S. Yang, T. Goel, E. P. Xing, T. Dao, and Y. Kim, “Log-linear attention,” 2026. [Online]. Available: <https://arxiv.org/abs/2506.04761>
- [31] Y. Zuo, Y. Yin, Z. Zeng, A. Li, B. Zhu, and Z. Wang, “Local linear attention: An optimal interpolation of linear and softmax attention for test-time regression,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.01450>
- [32] K. Team, Y. Zhang, Z. Lin, X. Yao, J. Hu, F. Meng, C. Liu, X. Men, S. Yang, Z. Li, W. Li, E. Lu, W. Liu, Y. Chen, W. Xu, L. Yu, Y. Wang, Y. Fan, L. Zhong, E. Yuan, D. Zhang, Y. Zhang, T. Y. Liu, H. Wang, S. Fang, W. He, S. Liu, Y. Li, J. Su, J. Qiu, B. Pang, J. Yan, Z. Jiang, W. Huang, B. Yin, J. You, C. Wei, Z. Wang, C. Hong, Y. Chen, G. Chen, Y. Wang, H. Zheng, F. Wang, Y. Liu, M. Dong, Z. Zhang, S. Pan, W. Wu, Y. Wu, L. Guan, J. Tao, G. Fu, X. Xu, Y. Wang, G. Lai, Y. Wu, X. Zhou, Z. Yang, and Y. Du, “Kimi linear: An expressive, efficient attention architecture,” 2025. [Online]. Available: <https://arxiv.org/abs/2510.26692>
- [33] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, D. Y. Fu, Z. Xie, B. Chen, C. Barrett, J. E. Gonzalez, P. Liang, C. Ré, I. Stoica, and C. Zhang, “Flexgen: High-throughput generative inference of large language models with a single gpu,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.06865>
- [34] Z. Zhang, S. Liu, R. Chen, B. Kailkhura, B. Chen, and Z. Wang, “Q-hitter: A better token oracle for efficient llm inference via sparse-quantized kv cache,” in *Proceedings of Machine Learning and Systems*, P. Gibbons, G. Pekhimenko, and C. D. Sa, Eds., vol. 6, 2024, pp. 381–394. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2024/file/bbb7506579431a85861a05fff048d3e1-Paper-Conference.pdf
- [35] H. Sun, L.-W. Chang, W. Bao, S. Zheng, N. Zheng, X. Liu, H. Dong, Y. Chi, and B. Chen, “Shadowkv: Kv cache in shadows for high-throughput long-context llm inference,” 2025. [Online]. Available: <https://arxiv.org/abs/2410.21465>
- [36] D. Yao, B. Shen, Z. Lin, W. Liu, J. Luan, B. Wang, and W. Wang, “Tailorkv: A hybrid framework for long-context inference via tailored kv cache optimization,” 2025. [Online]. Available: <https://arxiv.org/abs/2505.19586>